# Arrays

- ## what is an array in c?

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

All elements within a single array must be of the identical data type (e.g., all integers, all characters, all floats).

To create an array, define the data type (like int) and specify the name of the array followed by square brackets [].

To insert values to it, use a comma-separated list inside curly braces, and make sure all values are of the same data type:

# Declaration:

- The basic syntax for declaring a one-dimensional array is:

```
data_type array_name[size];
```

```
int numbers[5]; // Declares an integer array named 'numbers' that can hold 5 elements.
```

```
char name[20];  // Declares a character array named 'name' that can hold 20 characters.
```

# Initialization:

- You can initialize an array when you declare it by providing a list of values enclosed in curly braces {}.

int numbers[5] = {10, 20, 30, 40, 50}; // Initializes 'numbers' with 5 values.

float temperatures[] = {98.6, 99.2, 97.8}; // Size is automatically determined (3 in this case).

char vowels[5] = {'a', 'e', 'i'}; // Partial initialization; remaining elements are set to null ('\0').

int scores[10] = {0}; // All 10 elements are initialized to 0.

# Initialization after Declaration:

- You can assign values to individual array elements after declaration using their index. Array indices in C start from 0.

```
int numbers[5]; // Declared

numbers[0] = 10;

numbers[1] = 20;// ... and so on
```

# Using Loops for Initialization:

- For larger arrays, loops are often used to initialize elements, especially if the values follow a pattern.

```
int arr[5];

for (int i = 0; i < 5; i++) {

arr[i] = i * 2; // Initializes elements with values 0, 2, 4, 6, 8

}
```

# Multidimensional Arrays

- A multidimensional array is basically an array of arrays.
- Arrays can have any number of dimensions. In this chapter, we will introduce the most common; two-dimensional arrays (2D).
- A 2D array is also known as a matrix (a table of rows and columns).
- int matrix[2][3] = {  {1, 4, 2}, {3, 6, 8}  };
- The first dimension represents the number of rows [2], while the second dimension represents the number of columns [3]. The values are placed in row-order, and can be visualized like this:

| | COLUMN 0 | COLUMN 1 | COLUMN 2 |
|---|---|---|---|
| ROW 0 | 1 | 4 | 2 |
| ROW 1 | 3 | 6 | 8 |

# Access the Elements of a 2D Array

- To access an element of a two-dimensional array, you must specify the index number of both the row and column.
- This statement accesses the value of the element in the first row (0) and third column (2) of the matrix array.
- Example
- int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

printf("%d", matrix[0][2]);  // Outputs 2

| | COLUMN 0 | COLUMN 1 | COLUMN 2 |
|---|---|---|---|
| ROW 0 | 1 | 4 | 2 |
| ROW 1 | 3 | 6 | 8 |

# Change Elements in a 2D Array

- To change the value of an element, refer to the index number of the element in each of the dimensions:
- The following example will change the value of the element in the first row (0) and first column (0):
- Example
- int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

  matrix[0][0] = 9;

  printf("%d", matrix[0][0]);  // Now outputs 9 instead of 1

# Loop Through a 2D Array

- To loop through a multi-dimensional array, you need one loop for each of the array's dimensions.
- The following example outputs all elements in the matrix array:
- Example
- int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

```
int i, j;
for (i = 0; i < 2; i++) {
  for (j = 0; j < 3; j++) {
    printf("%d\n", matrix[i][j]);
  }
}
```

| | COLUMN 0 | COLUMN 1 | COLUMN 2 |
|---|---|---|---|
| ROW 0 | 1 | 4 | 2 |
| ROW 1 | 3 | 6 | 8 |

# Three-Dimensional Arrays

- You can also declare arrays with more than two dimensions:

Example

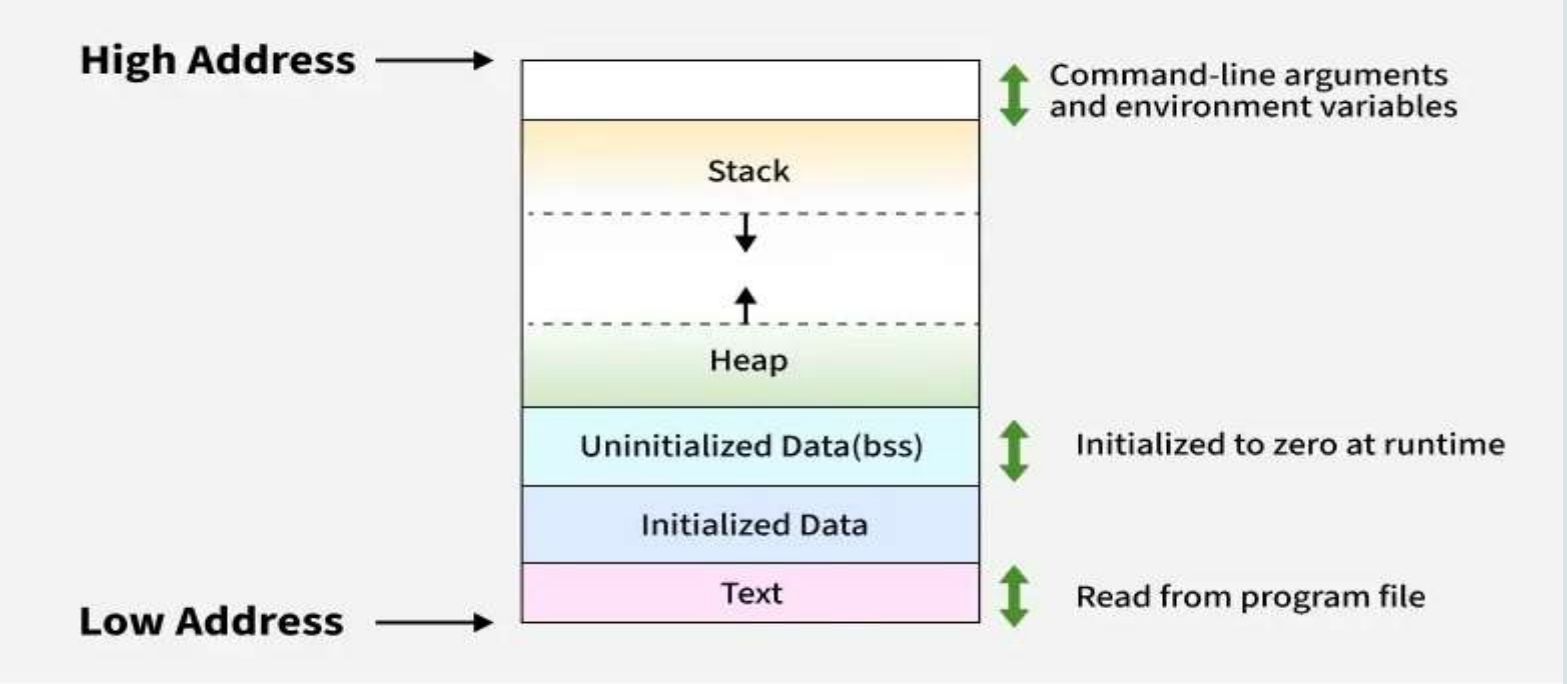
```
int example[2][4][3]; // A 3D array with 2 blocks, each with 4 rows and 3 columns
```

When to Use Multidimensional Arrays:

Multidimensional arrays are useful when your data is arranged in rows and columns, like a table, grid, or matrix.

# Memory Layout of C Programs

- The memory layout of a program refers to how the program's data is stored in the computer memory during its execution. Understanding this layout helps developers manage memory more efficiently and avoid issues such as segmentation faults and memory leaks.




*Different Segments in C Program's Memory*

# Continue..

- **Text Segment (.text):**
- This segment stores the compiled machine code instructions of the program. It is typically read-only and often shared among multiple instances of the same program to save memory.
- **Initialized Data Segment (.data):**
- This segment stores global and static variables that have been explicitly initialized with a non-zero value in the source code. It is a read-write segment, allowing the values of these variables to be modified during program execution.
- **Uninitialized Data Segment (.bss):**
- Also known as the Block Started by Symbol (BSS) segment, this area stores global and static variables that are not explicitly initialized. The operating system initializes these variables to zero before the program starts execution. Like the initialized data segment, it is read-write.

# Continue..

- **Heap:**
- This segment is used for dynamic memory allocation. Memory can be allocated and deallocated during program runtime using functions like malloc(), calloc(), realloc(), and free(). The heap grows upwards in memory addresses.
- **Stack:**
- This segment is used for managing function calls and storing local variables, function arguments, and return addresses. It operates as a Last-In, First-Out (LIFO) data structure. Each time a function is called, a new stack frame is pushed onto the stack, and when the function returns, its stack frame is popped off. The stack typically grows downwards in memory addresses.